Machine Learning for Finance

A Machine Learning Algorithm for Finite-Horizon Stochastic Control Problems in Economics (2024)

BAGOT Rémi, CHARDIGNY Lola, LETACONNOUX Lauryn

April 2026

1 Abstract

2 Why MMCC was introduced

The article [1] introduces the MMCC algorithm to address three major difficulties that arise in stochastic control problems. First, many relevant problems in economics and finance are formulated over a finite horizon and are time inhomogeneous, so that the optimal policy depends explicitly on the date and the problem becomes more difficult than in the stationary infinite-horizon setting. Second, these problems are often high dimensional and may involve complex state dynamics, which makes classical numerical methods quickly impractical because of the curse of dimensionality. Third, when the utility function or the objective is not time separable, a Bellman equation may no longer be available, which weakens the relevance of standard dynamic programming approaches.
In this setting, MMCC is proposed as a simulation based algorithm that can handle finite horizon, time inhomogeneous, and high dimensional stochastic control problems without relying on the Bellman equation. The idea is to approximate the optimal policies directly with neural networks and to update them sequentially in a backward direction. The authors emphasize the fact that the method results in a monotonic improvement of the objective function at each iteration, which supports good convergence properties.

3 The MMCC Algorithm

The Machine Learning Monte Carlo Control (MMCC) algorithm is an iterative procedure designed to solve stochastic control problems by combining Monte Carlo simulation and neural network optimization.

3.1 MMCC Algorithm in time-separable case

In this case we consider the utility function u is time-separable.

Initialization: Set $k = 1$ and initialize the vector of control parameters

$$x^0 = (c_0^0, \theta_1^0, \theta_2^0, \dots, \theta_{T-1}^0).$$

The control policy is parameterized by a neural network of the form

$$c(t, s_t, \theta_t) = \sigma_K \circ T_K \circ \cdots \circ \sigma_1 \circ T_1.$$

Iteration: ($k - th$ iteration) Repeat the following steps until a stopping criterion is satisfied.

Step A: Monte Carlo simulation Given the current parameter vector

$$x^{k-1} = (c_0^{k-1}, \theta_1^{k-1}, \dots, \theta_{T-1}^{k-1}),$$

simulate N i.i.d. sample paths of the state process.
For each trajectory $l = 1, \dots, N$, set $s_{0,l}^k = s_0$ and recursively compute

$$c_{t,l}^k = c(t, s_{t,l}^k, \theta_t^{k-1}), \quad s_{t+1,l}^k = \psi_{t+1}(s_{t,l}^k, c_{t,l}^k, z_{t+1,l}), \quad t = 0, \dots, T-1.$$

The N simulated paths are then partitioned into m minibatches of size b, such that $N = mb$.

Step B: Backward optimization of the policy parameters: For $t = T-1, T-2, \dots, 1$, the parameter θ_t is updated by solving a sequence of stochastic optimization problems.
For each minibatch $i = 1, \dots, m$, we maximize the following expectation:

$$\begin{aligned} &\max_{\theta_t} \quad \mathbb{E}_0 \left[\sum_{j=t}^{T-1} u_{j+1}(s_{j+1}, s_j, c_j) \;\middle|\; c_0^{k-1}, \theta_1^{k-1}, \dots, \theta_{t-1}^{k-1}, \theta_t, \theta_{t+1}^k, \dots, \theta_{T-1}^k \right] \\ &= \max_{\theta_t} \quad \mathbb{E}_0 \Bigg\{ \frac{1}{b} \sum_{l=1}^{b} \Bigg[u_{t+1}\big(s_{t+1,l,i}^k(\theta_t),\, s_{t,l,i}^k,\, c_{t,l,i}^k(\theta_t)\big) \\ &\qquad\qquad + \sum_{j=t+1}^{T-1} u_{j+1}\big(s_{j+1,l,i}^k(\theta_t),\, s_{j,l,i}^k(\theta_t),\, c_{j,l,i}^k(\theta_t)\big) \Bigg] \Bigg\}. \end{aligned} \tag{1}$$

where $c_{t,l,i}^k(\theta_t) = c(t, s_{t,l,i}^k, \theta_t)$ and the sequence

$$\big(s_{t+1,l,i}^k(\theta_t),\, c_{t+1,l,i}^k(\theta_t),\, \dots,\, s_{T-1,l,i}^k(\theta_t),\, c_{T-1,l,i}^k(\theta_t),\, s_{T,l,i}^k(\theta_t)\big)$$

is the simulated trajectory starting from $s_{t,l,i}^k$ and follows the control parameters $\theta_t, \theta_{t+1}^k, \dots, \theta_{T-1}^k$:

- $s_{t+1,l,i}^k(\theta_t) = \psi(s_{t,l,i}^k, c_{t,l,i}^k(\theta_t), z_{t+1,l,i})$
- $c_{t+1,l,i}^k(\theta_t) = c(t+1, s_{t+1,l,i}^k(\theta_t), \theta_{t+1}^k)$
- $s_{t+2,l,i}^k(\theta_t) = \psi(s_{t+1,l,i}^k(\theta_t), c_{t+1,l,i}^k(\theta_t), z_{t+2,l,i})$
- $c_{t+2,l,i}^k(\theta_t) = c(t+2, s_{t+2,l,i}^k(\theta_t), \theta_{t+2}^k)$
- $\dots$

The parameter θ_t is updated using the **Adam optimization algorithm**. After processing all minibatches, we obtain the optimal updated parameter θ_t^k.
*The total computational cost of **Step A** is $\mathcal{O}(m(T-t+1))$.*

Step C: Optimization of the initial control Finally, the initial control c_0 is updated using a similar Monte Carlo approximation.
For each minibatch $i = 1, \dots, m$, we consider the following conditional expectation:

$$\begin{aligned} &\max_{c_0} \quad \mathbb{E}_0 \left[\sum_{j=0}^{T-1} u_{j+1}(s_{j+1}, s_j, c_j) \;\middle|\; c_0, \theta_1^k, \dots, \theta_{T-1}^k \right] \\ &= \max_{c_0} \quad \mathbb{E}_0 \Bigg\{ \frac{1}{b} \sum_{l=1}^{b} \Bigg[u_1\big(s_{1,l,i}^k(c_0),\, s_0,\, c_0\big) + \sum_{j=1}^{T-1} u_{j+1}\big(s_{j+1,l,i}^k(c_0),\, s_{j,l,i}^k(c_0),\, c_{j,l,i}^k(c_0)\big) \Bigg] \Bigg\}. \end{aligned} \tag{2}$$

Where

$$\left\{s^k_{1,l,i}(c_0),\, c^k_{1,l,i}(c_0),\, \ldots,\, s^k_{T-1,l,i}(c_0),\, c^k_{T-1,l,i}(c_0),\, s^k_{T,l,i}(c_0)\right\}_{l=1}^{b}$$

is a sequence of sample paths that are simulated starting from s_0 and follows the control parameters: $(c_0, \theta^k_1, \ldots, \theta^k_{T-1})$.
The control c_0 is updated using the Adam algorithm, yielding c^k_0.
The total computational cost of ***Step C*** *is* $\mathcal{O}(mT)$.

Output from the iteration: The parameter vector is updated as

$$x^{k-1} \leftarrow x^k = (c^k_0, \theta^k_1, \theta^k_2, \ldots, \theta^k_{T-1}),$$

and the iteration counter is incremented, $k \leftarrow k+1$.
The total computational cost of each iteration of ***MMCC algorithm*** *is* $\mathcal{O}\big(\frac{m}{2}(T^2 - T)\big) = \mathcal{O}(mT^2)$.

3.2 MMCC algorithm in the non time-separable case

The MMCC algorithm is almost the same in the non time-separable case. The only difference lies in the expectations that are maximized in **Step B** and **Step C**.
In this case, the utility function u depends on the entire trajectory $(s_0, c_0, \ldots, s_T)$, rather than only on the future trajectory starting from s_t.

Step B: Backward optimization of the policy parameters For $t = T-1, T-2, \ldots, 1$, the parameter θ_t is updated by solving a sequence of stochastic optimization problems.
For each minibatch $i = 1, \ldots, m$, we maximize the following expectation:

$$\begin{aligned}
&\max_{\theta_t} \quad \mathbb{E}_0\left[u(s_0, c_0, s_1, c_1, \ldots, s_{T-1}, c_{T-1}, s_T, c_T) \;\middle|\; c^{k-1}_0, \theta^{k-1}_1, \ldots, \theta^{k-1}_{t-1}, \theta_t, \theta^k_{t+1}, \ldots, \theta^k_{T-1}\right] \\
&= \max_{\theta_t} \quad \mathbb{E}_0\left\{\frac{1}{b}\sum_{l=1}^{b}\left[u\big(s^k_{0,l,i}, c^k_{0,l,i}, \ldots, s^k_{t,l,i}, c^k_{t,l,i}(\theta_t), s^k_{t+1,l,i}(\theta_t), c^k_{t+1,l,i}(\theta_t), \ldots, s^k_{T,l,i}(\theta_t), c^k_{T,l,i}(\theta_t)\big)\big)\right]\right\}.
\end{aligned} \tag{3}$$

where $c^k_{t,l,i}(\theta_t) = c(t, s^k_{t,l,i}, \theta_t)$ and the sequence

$$\big(s^k_{t+1,l,i}(\theta_t),\, c^k_{t+1,l,i}(\theta_t),\, \ldots,\, s^k_{T-1,l,i}(\theta_t),\, c^k_{T-1,l,i}(\theta_t),\, s^k_{T,l,i}(\theta_t)\big)$$

is the simulated trajectory starting from $s^k_{t,l,i}$ and follows the control parameters $\theta_t, \theta^k_{t+1}, \ldots, \theta^k_{T-1}$:

- $s^k_{t+1,l,i}(\theta_t) = \psi(s^k_{t,l,i}, c^k_{t,l,i}(\theta_t), z_{t+1,l,i})$
- $c^k_{t+1,l,i}(\theta_t) = c(t+1, s^k_{t+1,l,i}(\theta_t), \theta^k_{t+1})$
- $s^k_{t+2,l,i}(\theta_t) = \psi(s^k_{t+1,l,i}(\theta_t), c^k_{t+1,l,i}(\theta_t), z_{t+2,l,i})$
- $c^k_{t+2,l,i}(\theta_t) = c(t+2, s^k_{t+2,l,i}(\theta_t), \theta^k_{t+2})$
- $\ldots$

$s^k_{0,l,i} = s_0$ and the sequence

$$\big(c^k_{0,l,i}, s^k_{1,l,i}, \ldots, c^k_{t,l,i}, s^k_{t,l,i},\big)$$

is already simulated in the **Step A** and follows the control parameters $\theta^{k-1}_1, \theta^{k-1}_2, \ldots, \theta^{k-1}_{t-1}$.
The parameter θ_t is updated using the **Adam optimization algorithm**. After processing all minibatches, we obtain the optimal updated parameter θ^k_t.

Step C: Optimization of the initial control Finally, the initial control c_0 is updated using a similar Monte Carlo approximation.
For each minibatch $i = 1, \dots, m$, we consider the following conditional expectation:

$$\begin{aligned} &\max_{c_0} \quad \mathbb{E}_0 \left[u(s_0, c_0, s_1, c_1, \dots, s_{T-1}, c_{T-1}, s_T, c_T) \;\middle|\; c_0, \theta_1^k, \dots, \theta_{T-1}^k \right] \\ = &\max_{c_0} \quad \mathbb{E}_0 \left\{ \frac{1}{b} \sum_{l=1}^{b} \left[u\big(s_{0,l,i}^k, c_0, s_{1,l,i}^k(c_0), c_{1,l,i}^k(c_0), \dots, s_{T,l,i}^k(c_0), c_{T,l,i}^k(c_0)\big)\big) \right] \right\}. \end{aligned} \tag{4}$$

Where $s_{0,l,i}^k = s_0$ and

$$\left\{ s_{1,l,i}^k(c_0),\ c_{1,l,i}^k(c_0),\ \dots,\ s_{T-1,l,i}^k(c_0),\ c_{T-1,l,i}^k(c_0),\ s_{T,l,i}^k(c_0) \right\}_{l=1}^{b}$$

is a sequence of sample paths that are simulated starting from s_0 and follows the control parameters: $(c_0, \theta_1^k, \dots, \theta_{T-1}^k)$.
The control c_0 is updated using the Adam algorithm, yielding c_0^k.

4 The Adam Algorithm [2]

Adam (AdaptiveMoment Estimation) is an algorithm for stochastic optimization.
A common optimization problem in machine learning is

$$\min_{\theta \in \Theta} L_n(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(Z_i, \theta).$$

Classical gradient descent has a cost of $\mathcal{O}(n)$ per iteration, and it is not suitable when data arrive in a continuous stream.
The idea of stochastic optimization is to use a subsample at each iteration, which leads in practice to convergence to a good optimum.

Stochastic Gradient Descent (SGD):

$$\theta^{(k+1)} = \theta^{(k)} - \alpha_k \nabla \ell(Z_{i_k}, \theta^{(k)})$$

where i_k is chosen uniformly at random in $\{1, \dots, n\}$. The cost per iteration is $\mathcal{O}(1)$ instead of $\mathcal{O}(n)$, but the method is noisy and can be unstable.

SGD with minibatch:

$$\bigcup_{k=1}^{K} B_k = [n], \quad \#B_k = m = \frac{n}{K}$$

$$\theta^{(k+1)} = \theta^{(k)} - \alpha_k \frac{1}{m} \sum_{j \in B_k} \nabla \ell(Z_j, \theta^{(k)})$$

This provides a better trade-off between variance and efficiency. The cost per iteration is $\mathcal{O}(m)$.

Momentum:

$$v^{(k+1)} = \beta v^{(k)} + (1 - \beta) \nabla \ell(Z_{i_k}, \theta^{(k)})$$
$$\theta^{(k+1)} = \theta^{(k)} - \alpha_k v^{(k+1)}$$

This method accumulates gradients from previous iterations. It reduces oscillations and accelerates convergence in narrow valleys.

RMSProp:

$$g^{(k+1)} = \gamma g^{(k)} + (1-\gamma)\left(\nabla \ell(Z_{i_k}, \theta^{(k)})\right)^2$$
$$\theta^{(k+1)} = \theta^{(k)} - \frac{\alpha_k}{\sqrt{g^{(k+1)}} + \varepsilon} \nabla \ell(Z_{i_k}, \theta^{(k)})$$

This method adapts the learning rate for each coordinate.

Adaptive Moment Estimation (Adam): The Adam algorithm combines momentum and adaptive learning rates.

$$m^{(k+1)} = \beta_1 m^{(k)} + (1-\beta_1)\nabla \ell(Z_{i_k}, \theta^{(k)})$$
$$v^{(k+1)} = \beta_2 v^{(k)} + (1-\beta_2)\left(\nabla \ell(Z_{i_k}, \theta^{(k)})\right)^2$$
$$\hat{m}^{(k+1)} = \frac{m^{(k+1)}}{1-\beta_1^{k+1}}, \quad \hat{v}^{(k+1)} = \frac{v^{(k+1)}}{1-\beta_2^{k+1}}$$
$$\theta^{(k+1)} = \theta^{(k)} - \alpha_k \frac{\hat{m}^{(k+1)}}{\sqrt{\hat{v}^{(k+1)}} + \varepsilon}$$

Adam can also be combined with minibatches.

5 Specification of the Application

In order to illustrate the MMCC algorithm on a finance problem, we consider the dynamic hedging of an Asian option (path-dependent) under stochastic volatility models with proportional transaction costs.

5.1 The control problem

We consider a market with a stock S_t, with a stochastic volatility in the Heston model [3]:

$$dS_t = rS_t\,dt + \sqrt{V_t}\,S_t\,dW_t, \quad S_0 = s_0 \tag{5}$$
$$dV_t = \kappa(\theta - V_t)\,dt + \xi\sqrt{V_t}\left(\rho\,dW_t + \sqrt{1-\rho^2}\,d\widetilde{W}_t\right), \quad V_0 = v_0 \tag{6}$$

where r is the risk-free rate, $\kappa > 0$ is the mean-reversion speed, $\theta > 0$ is the long-run variance, $\xi > 0$ is the volatility of variance, W_t and $\widetilde{W}_t$ are two independent standard Brownian motions . We consider an Asian call options with strike K, whose payoff is

$$g(\bar{S}_T) = \left(\frac{1}{T}\int_0^T S_t\,dt - K\right)^+, \qquad \bar{S}_T = \frac{1}{T}\int_0^T S_t\,dt.$$

To embed the path dependence into the state vector, we introduce the auxiliary process:

$$A_0 = 0, \qquad dA_t = S_t\,dt,$$

so that

$$\bar{S}_T = \frac{A_T}{T}.$$

Let $\lambda > 0$ be the proportional transaction cost. Let Q_t be the stock position and B_t the cash account. Let $Q_0 = q_0$ be the initial stock position

When trading dQ_t at time t, the transaction cost is

$$\Lambda_t = \lambda S_t \, |dQ_t|.$$

The cash account evolves as

$$dB_t = rB_t \, dt - S_t \, dQ_t - \Lambda_t, \quad B_0 = b_0$$

The dynamic of the portfolio is

$$dY_t = dB_t + Q_t dS_t, \quad Y_0 = y$$

At maturity, the position is liquidated:

$$Y_T = B_T + Q_T S_T - \Lambda_T.$$

We consider the following stochastic control problem, we define the objective as the minimization of the terminal quadratic hedging error:

$$\min_{y,Q(.)} \mathbb{E}\Big[\Big(Y_T^{y,q,S,Q} - g(\bar{S}_T)\Big)^2\Big].$$

5.2 The discretized control problem

We discretize $[0,T]$ into $0 = t_0 < t_1 < \ldots < t_{N_T} = T$. The discretized control problem is

$$\min_{(y,q_0,\theta_{t_1},\ldots,\theta_{t_{N_T-1}})} \mathbb{E}\Big[\Big(Y_T^{y,q,S,Q} - g(\bar{S}_T)\Big)^2\Big]$$

subject to the dynamics

$$\begin{aligned}
S_{t_0} &= s_0, \quad V_{t_0} = v_0, \quad A_{t_0} = 0, \quad B_{t_0} = b_0, \quad Y_{t_0} = y, \quad Q_{t_0} = q_0, \\
Q_{t_n} &= c(t_n, S_{t_n}, V_{t_n}, A_{t_n}, B_{t_n}, Y_{t_n}, \theta_{t_n}), \\
S_{t_{n+1}} &= S_{t_n} \exp\left(\left(r - \frac{1}{2}V_{t_n}\right)(t_{n+1} - t_n) + \sqrt{V_{t_n}^+}\,(W_{t_{n+1}} - W_{t_n})\right), \\
V_{t_{n+1}} &= V_{t_n} + \kappa(\theta - V_{t_n})(t_{n+1} - t_n) + \xi\sqrt{V_{t_n}^+}\left(\rho\,(W_{t_{n+1}} - W_{t_n}) + \sqrt{1-\rho^2}\,(\widetilde{W}_{t_{n+1}} - \widetilde{W}_{t_n})\right), \\
A_{t_{n+1}} &= A_{t_n} + S_{t_{n+1}}(t_{n+1} - t_n), \\
B_{t_{n+1}} &= e^{r(t_{n+1}-t_n)}\left(B_{t_n} - (Q_{t_n} - Q_{t_{n-1}})S_{t_n} - \lambda S_{t_n}|Q_{t_n} - Q_{t_{n-1}}|\right), \\
Y_{t_{n+1}} &= Y_{t_n} + (B_{t_{n+1}} - B_{t_n}) + Q_{t_n}(S_{t_{n+1}} - S_{t_n}), \\
Y_{t_N} &= B_{t_N} + Q_{t_{N-1}} S_{t_N} - \lambda S_{t_N}|Q_{t_{N-1}}|.
\end{aligned}$$

The initial position q_0 is chosen as an approximation of the initial optimal hedging strategy. More generally, the control

$$c(t_n, S_{t_n}, V_{t_n}, A_{t_n}, B_{t_n}, Y_{t_n}, \theta_{t_n})$$

is used as a neural-network approximation of the optimal hedging strategy at time t_n.

5.3 For the implementation of the MMCC algorithm

The global vector of control parameters is

$$x = (y, q_0, \theta_{t_1}, \ldots, \theta_{t_{N_T-1}}),$$

where y is the initial capital and q_0 is the initial stock position. For each date t_n, $n = 1, \ldots, N-1$. The state vector is

$$\xi_n = (S_{t_n}, V_{t_n}, A_{t_n}, B_{t_n}, Y_{t_n})$$

The rebalancing decision is :

$$Q_{t_n} = c(t_n, \xi_n, \theta_{t_n})$$

The transition from ξ_{t_n} to ξ_{t_n+1} is therefore given by

$$\xi_{n+1} = \psi_{n+1}(\xi_{t_n}, Q_{t_n}, z_{n+1}),$$

where $z_{n+1} \in \mathbb{R}^{n_z}$ is a random vector denoting the random shock in the $(t+1)th$ period, and ψ_{n+1} is the states evolution fucntion.

5.4 Choice of the neural-network architecture

Following the general MMCC framework, we use one feed-forward neural network per trading date,

$$q_n = c(t_n, s_n, \theta_n), \qquad n = 1, \ldots, N-1,$$

where the input is the state vector

$$s_n = (S_n, V_n, A_n, q_{n-1}, B_n) \in \mathbb{R}^5,$$

and the output is the scalar stock position $q_n \in \mathbb{R}$.
Because our state dimension is moderate, we choose a compact architecture. Our baseline specification is:

$$5 \longrightarrow 64 \longrightarrow 64 \longrightarrow 1.$$

In other words, for each date t_n, the control policy is represented by a neural network with:

- an input layer of size 5,
- two hidden layers of size 64,
- one scalar output layer.

The hidden layers use the ReLU activation function:

$$\mathrm{ReLU}(x) = \max(x, 0),$$

and the output layer is linear:

$$q_n = W_3 \,\mathrm{ReLU}\big(W_2 \,\mathrm{ReLU}(W_1 s_n + b_1) + b_2\big) + b_3.$$

If one wishes to impose position constraints, for example $q_n \in [-q_{\max}, q_{\max}]$, the output layer can be replaced by a scaled hyperbolic tangent:

$$q_n = q_{\max} \tanh\big(W_3 \,\mathrm{ReLU}\big(W_2 \,\mathrm{ReLU}(W_1 s_n + b_1) + b_2\big) + b_3\big)\,.$$

The same architecture is used at every date, but with different parameters θ_n. Hence the global vector of control parameters is

$$x = (c_0, \theta_1, \ldots, \theta_{N-1}), \qquad c_0 = (y, q_0).$$

In the numerical implementation, the network parameters are updated with the Adam optimizer within each MMCC backward step. A natural first set of hyperparameters is:

- learning rate: 10^{-3} or 5×10^{-4},
- minibatch size: 128 or 256,
- early stopping based on the out-of-sample hedging loss.

If the baseline architecture underfits, one may increase the width to

$$5 \longrightarrow 128 \longrightarrow 128 \longrightarrow 1.$$

On the contrary, if the optimization becomes unstable, one may reduce the width or add gradient clipping.

References

[1] X. Peng, S. Kou and L. Zhang, *A Machine Learning Algorithm for Finite-Horizon Stochastic Control Problems in Economics*, 2024.

[2] Arnak Dalalyan, *Course Applied Statistical Learning, CREST, ENSAE Paris, IP Paris*, 2025.

[3] Steven L. Heston, *A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options*, 1993.